CS315 · ALGORITHMS ANALYSIS & DESIGN

Practice 2 Hash Tables Solutions

Practice on Lecture 3 · 9 questions

Muharib

01 Solution Q1: chaining

Key	22	1	13	11	24	33	18	42	31
k mod 11	0	1	2	0	2	0	7	9	9

Index	Chain
0	22 → 11 → 33
1	1
2	13 → 24
3	∅
4	∅
5	∅
6	∅
7	18
8	∅
9	42 → 31
10	∅

`22, 11, 33` all leave remainder 0 when divided by 11, so they go to cell 0 in the order they arrive.

02 Solution Q2: linear probing

Key	k mod 11	Probes	Cell
22	0	0	**0**
1	1	1	**1**
13	2	2	**2**
11	0	0 → 1 → 2 → 3	**3**
24	2	2 → 3 → 4	**4**
33	0	0 → 1 → 2 → 3 → 4 → 5	**5**
18	7	7	**7**
42	9	9	**9**
31	9	9 → 10	**10**

0	1	2	3	4	5	6	7	8	9	10
22	1	13	11	24	33		18		42	31

Key 33 needed the most: 6 probes (cells 0 to 5), because the items lumped together at the start of the table. This is the weakness of linear probing.

03 Solution Q3: double hashing

Key	k mod 13	7 − (k mod 7)	Probes	Cell
14	1	7	1	**1**
27	1	1	1 → 2	**2**
40	1	2	1 → 3	**3**
19	6	2	6	**6**
53	1	3	1 → 4	**4**
7	7	7	7	**7**
66	1	4	1 → 5	**5**

0	1	2	3	4	5	6	7	8	9	10	11	12
	14	27	40	53	66	19	7					

```
27:  h = 1 taken,  d = 7 − (27 mod 7) = 7 − 6 = 1   →  (1 + 1) mod 13 = 2
```

```
40:  h = 1 taken,  d = 7 − (40 mod 7) = 7 − 5 = 2   →  (1 + 2) mod 13 = 3
```

```
53:  h = 1 taken,  d = 7 − (53 mod 7) = 7 − 4 = 3   →  (1 + 3) mod 13 = 4
```

```
66:  h = 1 taken,  d = 7 − (66 mod 7) = 7 − 3 = 4   →  (1 + 4) mod 13 = 5
```

Five keys share `h(k) = 1`, but each has a different step `d(k)`, so each one found its cell on the second probe.

04 Solution Q4: MAD

Key	10	22	31	4	15	28	17	88	59
(2k+5) mod 11	3	5	1	2	2	6	6	5	2

```
h(10) = (20 + 5) mod 11 = 25 mod 11 = 3
```

```
h(88) = (176 + 5) mod 11 = 181 mod 11 = 5
```

a) Chaining:

Index	Chain
0	ø
1	31
2	4 → 15 → 59
3	10
4	ø
5	22 → 88
6	28 → 17
7	ø
8	ø
9	ø
10	ø

b) Linear probing:

Key	(2k+5) mod 11	Probes	Cell
10	3	3	**3**
22	5	5	**5**
31	1	1	**1**
4	2	2	**2**
15	2	2 → 3 → 4	**4**
28	6	6	**6**
17	6	6 → 7	**7**
88	5	5 → 6 → 7 → 8	**8**
59	2	2 → 3 → 4 → 5 → 6 → 7 → 8 → 9	**9**

0	1	2	3	4	5	6	7	8	9	10
	31	4	10	15	22	28	17	88	59	

05 Solution Q5: the lecture example

Key	12	44	13	55	5
(3i+2) mod 7	3	1	6	6	3

Index	Chain
0	∅
1	44
2	∅
3	12 → 5
4	∅
5	∅
6	13 → 55

06 Solution Q6: removal and AVAILABLE

a) Search for 44: `44 mod 13 = 5`. Cell 5 holds 18, so we move to cell 6 and find it. We replace it with `AVAILABLE` (written AV in the table).

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	AV	59	32	22	31	73	

```
b)   findElement(31):   31 mod 13 = 5

     probes 5 → 6 → 7 → 8 → 9 → 10

     found at cell 10, returns its element

c)   findElement(70):   70 mod 13 = 5

     probes 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12

     cell 12 is empty, returns NO_SUCH_KEY

d)   insertItem(57):    57 mod 13 = 5

     probes 5 → 6

     cell 6 stores AVAILABLE, so 57 is stored in cell 6
```

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	57	59	32	22	31	73	

The key point

A search does not stop at `AVAILABLE`. It stops only at an empty cell. An insertion stops at either.

07 Solution Q7: load factor

```
a)   α = N / M = 8 / 13 ≈ 0.615
```

```
     Expected probes = 1 / (1 − α) = 1 / (5/13) = 13 / 5 = 2.6
```

```
b)   α = 0.9:   1 / (1 − 0.9) = 10 probes
```

Conclusion: the closer the load factor gets to 100%, the more the number of probes grows. Hashing is fast provided the table is not nearly full.

c) Rehashing: we create a new table of double the size and move the items into it, which lowers the load factor.

08 Solution Q8: compression map

```
h₂(2) = (28 + 3) mod 7  = 31 mod 7  = 3
```

```
h₂(5) = (70 + 3) mod 7  = 73 mod 7  = 3
```

```
h₂(9) = (126 + 3) mod 7 = 129 mod 7 = 3
```

b) Every key maps to cell `b = 3`, because `a mod N = 14 mod 7 = 0`. This violates the MAD condition `a mod N ≠ 0`.

09 Solution Q9: short answers

#	Answer
a	1) Chaining: each cell points to a linked list of the elements mapped there. It is simple but needs memory outside the table. 2) Linear probing (open addressing): place the item in the next available cell, circularly. Its weakness is lumping. 3) Double hashing (open addressing): jump by a step `d(k)` from a second function, `(i + j·d(k)) mod N`.
b	When it disperses the keys over the cells in an apparently random, even way, so collisions are few, and it is fast to compute.
c	insertItem takes `O(1)`. findElement and removeElement take `O(n)`.
d	The worst case is `O(n)`, and it occurs when all the inserted keys collide. The expected time is `O(1)`.
e	Polynomial accumulation. The component sum is not good because words with the same letters in a different order give the same sum.
f	To allow probing of all the cells of the table. `d(k)` can never be zero.